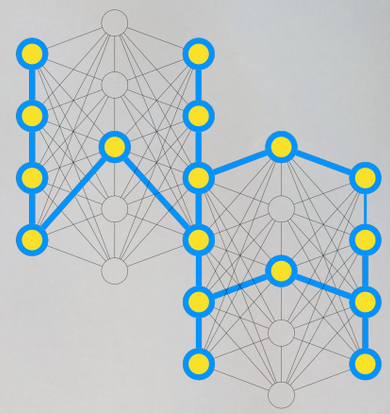


# jQuery

## Web Applications

Master Degree in Computer Engineering
Master Degree in Cybersecurity
Master Degree in ICT for Internet and Multimedia
Academic Year 2023/2024

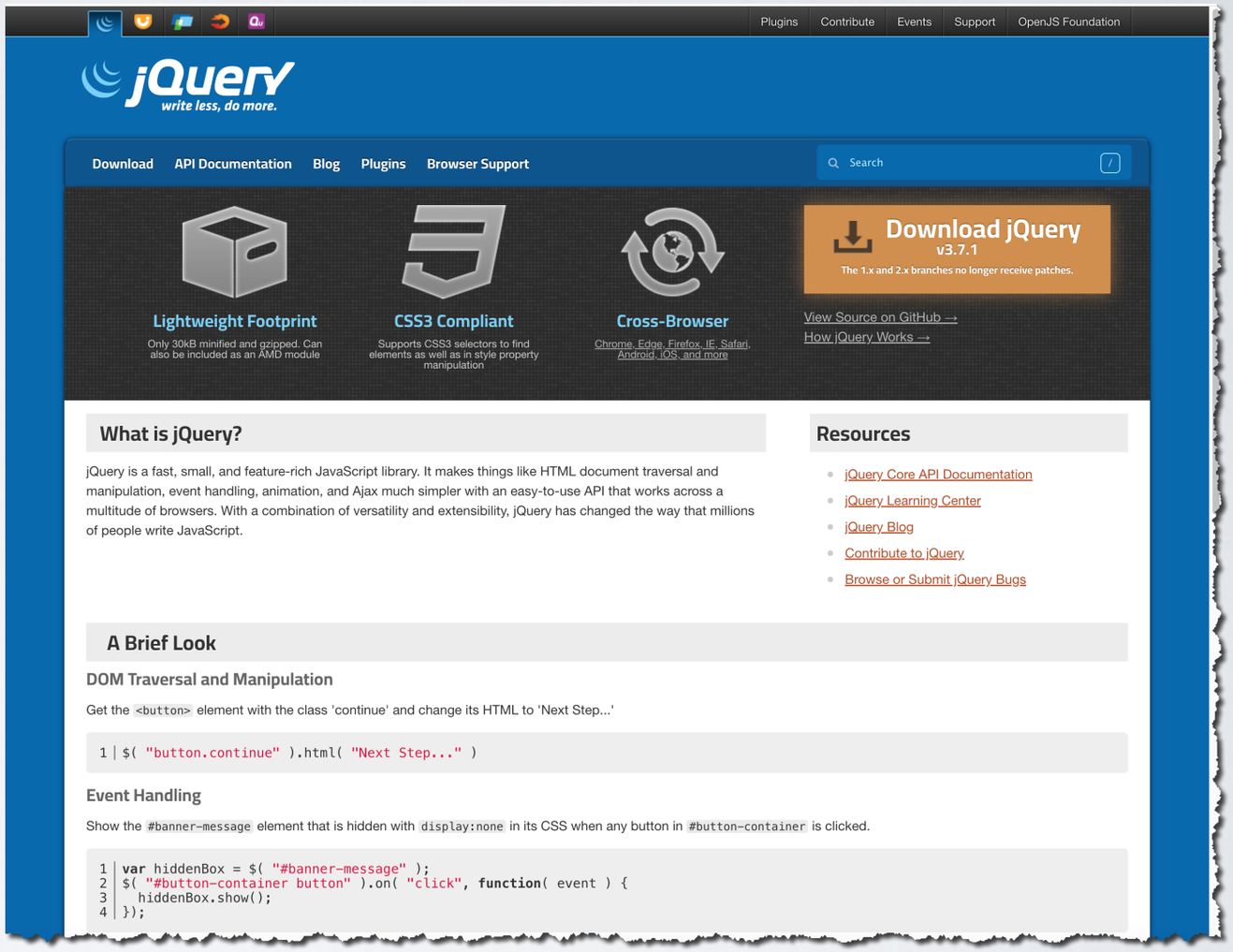

**Nicola Ferro**

Intelligent Interactive Information Access (IIIA) Hub
Department of Information Engineering
University of Padua

# Introduction to jQuery

# jQuery

- jQuery is a fast, small, and feature-rich JavaScript library (http://jquery.com/).
- jQuery offers a simple way to achieve a variety of common JavaScript tasks quickly and consistently across all major browsers and without any fallback code needed.
- It allows to:
  - Select elements in a simpler and more powerful way with CSS-style selectors;
  - Manipulate the DOM tree;
  - Attach event listeners without any fallback code.



# Why jQuery?

- jQuery is a lightweight, “write less, do more”, JavaScript library. Its aim is to make easier to use JS on a website.
- It allows to perform many tasks, that otherwise would have required many lines of JavaScript code, in single lines of code.
- There are many other JS libraries available. jQuery is one of the most popular and extendable.
- Some of jQuery’s features are:
  - HTML/DOM manipulation
  - CSS manipulation
  - HTML event methods
  - Effect and animations
  - Ajax
  - Utilities

# jQuery Basics

- The jQuery library defines a single global function named `jQuery()`, with the symbol `$` as a shortcut for it.

```
var divs = $("div");
```

- The value returned by this function represents a set of zero or more DOM elements and is known as a jQuery object.
- jQuery objects define many methods for operating on the sets of elements they represent.

```
$("p.details").css("background-color", "yellow").show("fast");
```

# jQuery Objects

jQuery objects are array-like and they have the following properties:

- The `length` property;
- The `selector` property is the selector string (if any) that was used when the jQuery object was created.
- The `context` property is the context object that was passed as the second argument to $(), or the Document object otherwise.
- The `jquery` property: testing for the existence of this property is a simple way to distinguish jQuery objects from other array-like objects.

# Queries and Query Results

- When you pass a CSS selector string to `$()`, it returns a jQuery object that represents the set of matched elements.
- jQuery objects are array-like: they have a **`length`** property and you can access the contents of the jQuery object using standard square-bracket array notation:

```
$("body").length
$("body")[0]
```

- If you prefer not to use array notation with jQuery objects, you can use the **`size()`** method instead of the length property and the **`get()`** method instead of indexing with square brackets. If you need to convert a jQuery object to a true array, call the **`toArray()`** method.

# Creating DOM Elements

If a string is passed as the parameter to `$()`, jQuery examines the string to see if it looks like HTML (i.e., it starts with <tag ... >). If not, the string is interpreted as a selector expression. But if the string is a HTML snippet, jQuery attempts to create new DOM elements, then a jQuery object is created and returned.

```
var img = $("<img/>",
          { src: url,
            css: {borderWidth:5},
            click: handleClick
          });
```

# Each() Method

- If you want to loop over all elements in a jQuery object, you can call the **`each()`** method instead of writing a for loop. The **`each()`** method is similar to the **`forEach()`** array method.
- It expects a callback function as its sole argument, and it invokes that callback function once for each element in the jQuery object.
- Despite the power of the **`each()`** method, it is not very commonly used, since jQuery methods usually iterate implicitly over the set of matched elements and operate on them all.

# jQuery Getters and Setters

# jQuery Getters and Setters

jQuery objects allow you to get or set the value of HTML attributes, CSS styles or element content:

- jQuery uses a single method as both getter and setter. If you pass a new value to the method, it sets that value; if you don't specify a value, it returns the current value.
- When used as setters, these methods set values on every element in the jQuery object, and then return the jQuery object to allow method chaining.
- When used as getters, these methods query only the first element of the set of elements and return a single value, therefore they can only appear at the end of a method chain.
- When used as setters, these methods often accept object arguments. In this case, each property of the object specifies a name and a value to be set.
- When used as setters, these methods often accept functions as values. In this case, the function is invoked to compute the value to be set.

# Getting and Setting HTML Attributes

The `attr()` method acts as both a getter and a setter.

`attr()` as setter

```
$("a").attr("href", "allMyHrefsAreTheSameNow.html");

$("a").attr({
    title: "all titles are the same too!",
    href: "somethingNew.html"
});
```

`attr()` as getter

```
$("a").attr("href");
```

# Getting and Setting CSS Attributes

- The `css()` method is similar to the `attr()` method, but it works with the CSS styles of an element.
- When querying style values, `css()` returns the current (or computed) style of the element: the returned value may come from the style attribute or from a stylesheet.

Setting CSS properties

```
$("h1").css("fontSize", "100px");
$("h1").css({
    fontSize: "100px",
    color: "red"
});
```

Getting CSS properties

```
$("h1").css("fontSize");
$("h1").css("font-size");
```

# Getting and Setting CSS Classes

- jQuery defines `addClass()` and `removeClass()` to add and remove classes from the selected elements.
- `toggleClass()` adds classes to elements that don't already have them and removes classes from those that do.
- `hasClass()` tests for the presence of a specified class.

```
var h1 = $("h1");
h1.addClass("big");
h1.removeClass("big");
h1.toggleClass("big");
if (h1.hasClass("big")) {
    ...
}
```

# Getting and Setting HTML Form Values

`val()` is a method for setting and querying the value attribute of HTML form elements and also for querying and setting the selection state of checkboxes, radio buttons, and <select> elements.

Setting the input value

```
$("input[type=text].tags").val(function(index, value) {
  return value.trim();
});
```

Getting input values

```
var singleValues = $("#single").val();
var multipleValues = $("#multiple").val()
```

# Getting and Setting Element Content

- The `text()` and `html()` methods query and set the plain-text or HTML content of an element or elements.
- When invoked with no arguments, `text()` returns the plain-text content of all descendant text nodes of all matched elements.
- If you invoke the `html()` method with no arguments, it returns the HTML content of just the first matched element.
- If you pass a string to `text()` or `html()`, that string will be used for the plain-text or HTML-formatted text content of the element, and it will replace all existing content.

# Altering the DOM Structure

# Inserting and Replacing Elements

- Each of the following methods takes an argument that specifies the content that is to be inserted into the document. This can be a string of plain text or of HTML to specify new content, or it can be a jQuery object or an Element or text Node.
- The insertion is made into or before or after or in place of (depending on the method) each of the selected elements.
- If the content to be inserted is an element that already exists in the document, it is moved from its current location. If it is to be inserted more than once, the element is cloned as necessary.
- These methods all return the jQuery object on which they are called.

# Inserting and Replacing Elements

| Operation | $(target).method(content) | $(content).method(target) |
|---|---|---|
| insert content at end of target | `append()` | `appendTo()` |
| insert content at start of target | `prepend()` | `prependTo()` |
| insert content after target | `after()` | `insertAfter()` |
| insert content before target | `before()` | `insertBefore()` |
| replace target with content | `replaceWith()` | `replaceAll()` |

# Inserting and Replacing Elements

```
$("#log").append("<br/>"+message);
$("p").prepend("<b>Hello </b>");;
$("h1").before("<hr/>");
$("h1").after("<hr/>");
$("hr").replaceWith("<br/>");
```

```
$("<br/>+message").appendTo("#log");
$(document.createTextNode("<b>Hello </b>")).prependTo("p");
$("<hr/>").insertBefore("h1");
$("<hr/>").insertAfter("h1");
$("<br/>").replaceAll("hr");
```

For more examples on these jQuery methods (and more): https://www.w3schools.com/jquery/jquery_ref_html.asp

# Copying Elements

- If you insert elements that are already part of the document, those elements will simply be moved, not copied, to their new location.
- If you are inserting the elements in more than one place, jQuery will make copies as needed.
- If you want to copy elements to a new location instead of moving them, you must first make a copy with the `clone()` method. clone() makes and returns a copy (jQuery object) of each selected element (and of all of the descendants of those elements).

# Copying Elements Example

```
<div class="container">
  <div class="hello">Hello</div>
  <div class="goodbye">Goodbye</div>
</div>


$(".hello").clone().appendTo(".goodbye");
```

```
<div class="container">
  <div class="hello">Hello</div>
  <div class="goodbye">
    Goodbye
    <div class="hello">Hello</div>
  </div>
</div>
```

# Wrapping Elements

- jQuery defines three wrapping functions.
  - `wrap()` wraps each of the selected elements.
  - `wrapInner()` wraps the contents of each selected element.
  - `wrapAll()` wraps the selected elements as a group.
- These methods are usually passed a newly created wrapper element or a string of HTML used to create a wrapper.

```
$("h1").wrap(document.createElement("i"));
$(".inner").wrapInner("<div class='new'></div>");
$(".inner").wrapAll("<div class=‘new'></div>");
```

# Deleting Elements

jQuery defines several methods for deleting elements.

- `empty()` removes all children of each of the selected elements.
- `remove()` removes the selected elements (together with their event handlers and data) from the document. If you pass an argument, that argument is treated as a selector, and only elements of the jQuery object that also match the selector are removed.
- `detach()` method works like remove() but does not remove event handlers and data. detach() may be more useful when you want to temporarily remove elements from the document for later reinsertion.
- `unwrap()` method performs element removal in a way that is the opposite of the wrap() or wrapAll() method: it removes the parent element of each selected element without affecting the selected elements or their siblings. That is, for each selected element, it replaces the parent of that element with its children.

# Handling Events with jQuery

# Simple Event Handler Registration

- jQuery defines simple event-registration methods for each of the commonly used and universally implemented browser events.
- To register an event handler for click events, for example, just call the `click()` method (nb: only the p being clicked is changed to gray):

```
$("p").click(function() { $(this).css("background-color", "gray"); });
```

- Calling a jQuery event-registration method registers your handler on all of the selected elements. This is typically much easier than one-at-a-time event handler registration with addEventListener().

# Event Handler Registration Methods

- `blur()`
- `change()`
- `click()`
- `dblclick()`
- `focus()`
- `focusin()`
- `focusout()`
- `error()`
- `keydown()`
- `keypress()`
- `keyup()`
- `load()`
- `mousedown()`
- `mouseenter()`
- `mouseleave()`
- `mousemove()`
- `mouseout()`
- `mouseover()`
- `mouseup()`
- `resize()`
- `scroll()`
- `select()`
- `submit()`
- `unload()`

# jQuery Event Handler

- The method `bind()` binds a handler for a named event type to each of the elements in the jQuery object. Using bind() allows you to use more advanced event registration features.
- `bind()` expects an event type string as its first argument and an event handler function as its second.

```
$("p").click(f);        $("p").bind("click", f);
```

- If the first argument is a space separated list of event types, then the handler function will be registered for each of the named event types.

```
$("a").hover(f);

$("a").bind("mouseenter mouseleave", f);
```

# Deregistering Event Handlers

- After registering an event handler with `bind()` (or with any of the simpler event registration methods), you can deregister it with `unbind()`.
- `unbind()` only deregisters event handlers registered with bind() and related jQuery methods (not with addEventListener()).
- With no arguments, `unbind()` deregisters all event handlers (each event for each element):

```
$("*").unbind();
```

- With string arguments, all handlers for the named event type are unbound from all elements in the jQuery object:

```
$("a").unbind("mouseover mouseout");
```

# AJAX with jQuery

# AJAX and jQuery

- jQuery provides several methods for AJAX functionality. With these, it is possible to request text, HTML, XML, or JSON from remote servers using both HTTP GET and POST.
- Writing regular AJAX code can be tricky, because different browsers have different syntax for AJAX implementation. Thus, it may be necessary to write extra code to test for different browsers. jQuery takes care of this.

# AJAX Function

- The `jQuery.ajax()` function performs asynchronous HTTP requests. It underlies all Ajax requests sent by jQuery. It is often unnecessary to directly call this function, as several higher-level alternatives are available.
- `ajax()` accepts a single argument: an options object whose properties specify the details about how the AJAX request is to be performed.
- By default, data passed in to the data option as an object will be processed and transformed into a query string, fitting to the default content-type `"application/x-www-form-urlencoded"`.

```
$.ajax({
  method: "POST",
  url: "some.jsp",
  data: { name: "John", location: "Boston" }
})
  .done(function( msg ) {
    alert( "Data Saved: " + msg );
  });
```

# AJAX Utility Functions - get()

- `jQuery.get()`, load data from the server using a HTTP GET request.
- GET is basically used for getting data from a server. It may also return cached data.

```
$.get(URL, callback);
```

- The required URL parameter specifies the URL we wish to request.
- The optional callback parameter is the name of a function to be executed if the request succeeds. The callback has two parameters: the content of the page requested, and the status of the request.

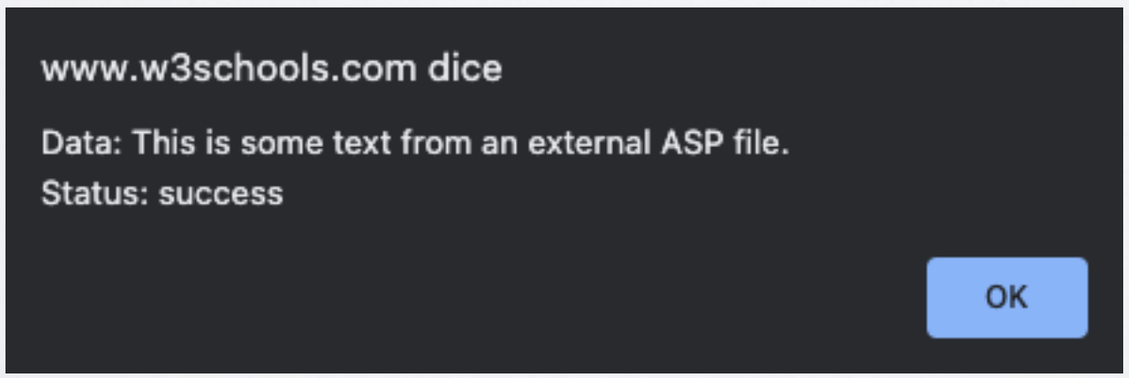
```
$.get("test.jsp", { name: "John", time: "2pm" } )
  .done(function(data, status) {
    alert("Data Loaded: " + data “\nStatus: ” + status);
  });
```

# AJAX Utility Functions - get()

```
<!DOCTYPE html>
<html>
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js">
</script>
<script>
$(document).ready(function(){
  $("button").click(function(){
    $.get("demo_test.asp", function(data, status){
      alert("Data: " + data + "\nStatus: " + status);
    });
  });
});
</script>
</head>
<body>

<button>Send an HTTP GET request to a page and get the result back</button>

</body>
</html>
```



# AJAX Utility Functions - post()

- `jQuery.post()`, load data from the server using a HTTP POST request.

```
$.post(URL, data, callback)
```

- URL specifies the URL we wish to request
- The optional data parameters specifies some data to send along with the request.
- The optional callback parameter is the name of a function to be executed if the request succeeds.

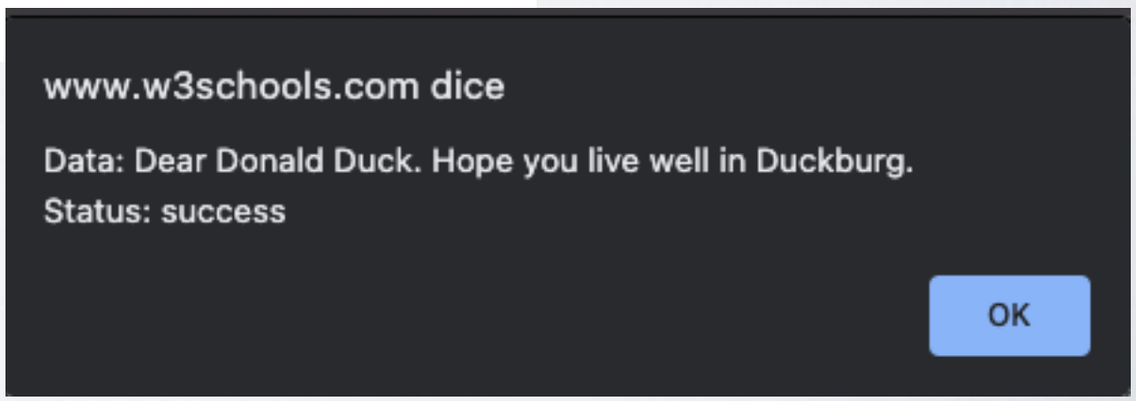
```
$.post("test.jsp", { name: "John", time: "2pm" })
  .done(function(data) {
    alert("Data Loaded:" + data);
  });
```

# AJAX Utility Functions - post()

```
<!DOCTYPE html>
<html>
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js">
</script>
<script>
$(document).ready(function(){
  $("button").click(function(){
    $.post("demo_test_post.asp",
    {
      name: "Donald Duck",
      city: "Duckburg"
    },
    function(data,status){
      alert("Data: " + data + "\nStatus: " + status);
    });
  });
});
</script>
</head>
<body>

<button>Send an HTTP POST request to a page and get the result back</button>

</body>
</html>
```



# AJAX Utility Functions - getScript()

`jQuery.getScript()`, load a JavaScript file from the server using a GET HTTP request, then execute it.

```
$.getScript("ajax/test.js", function( data, textStatus, jqxhr) {
  console.log(data); // Data returned
  console.log(textStatus); // Success
  console.log(jqxhr.status); // 200
  console.log("Load was performed.");
});
```

# AJAX Utility Functions - getJSON()

`jQuery.getJSON()`, load JSON-encoded data from the server using a GET HTTP request.

```
$.getJSON("ajax/test.json", function(data) {
  var items = [];
  $.each(data, function(key, val) {
    items.push( "<li id='" + key + "'>" + val + "</li>" );
  });

  $("<ul/>", {
    "class": "my-new-list",
    html: items.join( "" )
  }).appendTo( "body" );
});
```

# The load() Method

- `load()` is a simple but powerful AJAX method. It loads data from a server and puts it into a selected element. Its syntax:

```
$(selector).load(URL,data,callback);
```

- The `URL` parameter specifies the URL you want to load
- The `selector` specifies the elements where the returned data will be loaded
- The optional `data` parameter specifies a set of query-string key/value pairs to send along with the request.
- The optional `callback` parameter is the name of the *function* to be executed after the load() method is completed and the data are returned.

# The load() Method

- The `load()` method with an URL as argument will asynchronously load the content of that URL and then insert that content into each of the selected elements, replacing any content that is already there.

```
$("#result").load("ajax/test.html");
```

- The `load()` method, allows you to specify a fragment of the document to be inserted.

```
$("#result").load("ajax/test.html #container");
```

- The POST method is used if data is provided as an object; otherwise, GET is assumed.

```
$("#address").load("address.jsp", { zipcode:"02134", country:"IT" });
```

- An optional argument to load() is a callback function that will be invoked when the AJAX request completes successfully or unsuccessfully.

# Load text into a div (1)

```
<!DOCTYPE html>
<html>
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js">
</script>
<script>
$(document).ready(function(){
  $("button").click(function(){
    $("#div1").load("demo_test.txt");
  });
});
</script>
</head>
```

The anonymous function that uses the jQuery load method is put as callback of the click event on button elements. It loads the content of the demo_test.txt to the elements of id **`div1`**.

# Load text into a div (2)

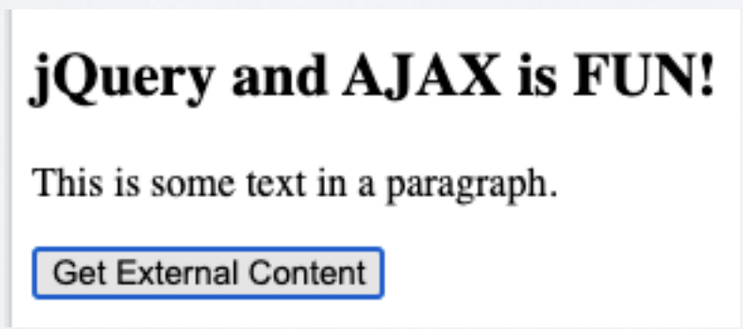
```
<body>

<div id="div1"><h2>Let jQuery AJAX Change This Text</h2></div>

<button>Get External Content</button>

</body>
```


After the click the content of div1 is *replaced.* In this example it is HTML directly injected in the page



# Load text into a div (3)

- It is possible to add a jQuery selector to the URL parameter to specify the part of the document to insert.

```
<script>
$(document).ready(function(){
  $("button").click(function(){
    $("#div1").load("demo_test.txt #p1");
  });
});
</script>
```


In this example, only the text contained in the paragraph of id p1 of the file demo_text.txt is inserted.

# Load text into a div (4)

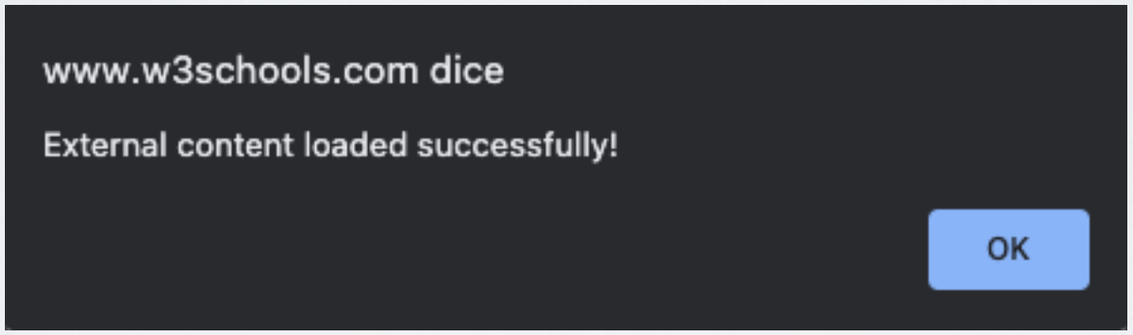
- The optional callback parameter specifies a callback function to run when the `load()` method is completed.
- This function can have different parameters:
  - `responseText` - contains the resulting content if the call succeeds
  - `statusTxt` - contains the status of the call
  - `xhr` - contains the XMLHttpRequest object

```
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js">
</script>
<script>
$(document).ready(function(){
  $("button").click(function(){
    $("#div1").load("demo_test.txt", function(responseTxt, statusTxt, xhr){
      if(statusTxt == "success")
        alert("External content loaded successfully!");
      if(statusTxt == "error")
        alert("Error: " + xhr.status + ": " + xhr.statusText);
    });
  });
});
</script>
```

# Load text into a div (5)


User clicks the button.

The callback function is invoked at the return of the data




Then the information is loaded into the page


These examples were taken from:

`https://www.w3schools.com/jquery/jquery_ajax_load.asp`

# Canvas

# Introduction to HTML5 Canvas

- The HTML5 specification includes the canvas element which gives you an easy and powerful way to draw graphics using JavaScript.
- For example it can be used to draw graphs, make photo compositions, create animations, or even do real-time video processing or rendering.
- For each canvas element you can use a "context" (similar to a page in a drawing pad), into which you can issue JavaScript commands to draw anything you want. Browsers can implement multiple canvas contexts and the different APIs provide the drawing functionality.

# The Canvas Element

```
<canvas id="tutorial" width="150" height="150"></canvas>
```

- The `<canvas>` element is similar to the <img> element, with the only clear difference being that it doesn't have the src and alt attributes.
- The `<canvas>` attributes `width` and `height` are both optional, when they are not specified the canvas will initially be 300 pixels wide and 150 pixels high.
- The element can be sized arbitrarily by CSS, but during rendering the image is scaled to fit its layout size: if the CSS sizing doesn't respect the ratio of the initial canvas, it will appear distorted.

# The Rendering Context

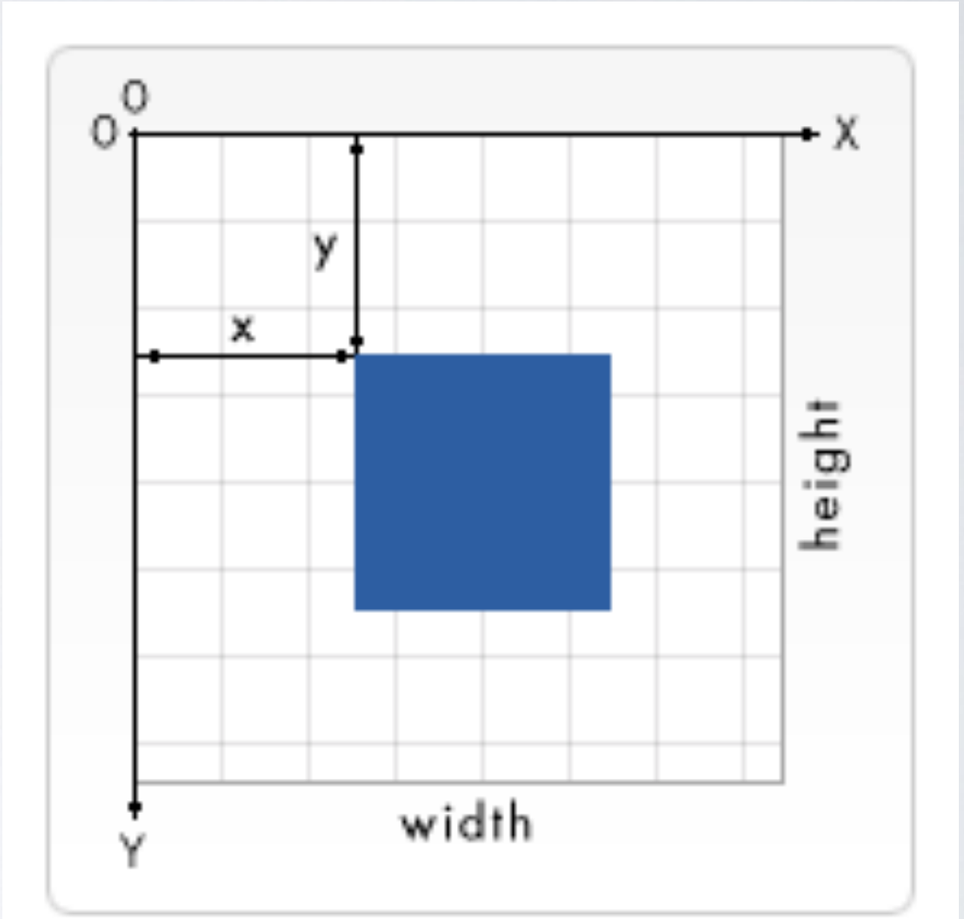
- The <canvas> element creates a fixed-size drawing surface that exposes one or more rendering contexts, which are used to create and manipulate the content shown.
- The canvas is initially blank. To display something, a script first needs to access the rendering context and draw on it.
- The <canvas> element has a method called `getContext()`, used to obtain the rendering context and its drawing functions.
- `getContext()` takes one parameter, the type of context. For 2D graphics you can specify "`2d`" to get a `CanvasRenderingContext2D`.

# Example

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8"/>
    <title>Canvas tutorial</title>
    <script type="text/javascript">
      function draw() {
        var canvas = document.getElementById('tutorial');
        if (canvas.getContext) {
          var ctx = canvas.getContext('2d');
        }
      }
    </script>
    <style type="text/css">
      canvas { border: 1px solid black; }
    </style>
  </head>
  <body onload="draw();">
    <canvas id="tutorial" width="150" height="150"></canvas>
  </body>
</html>
```

# The Canvas Grid

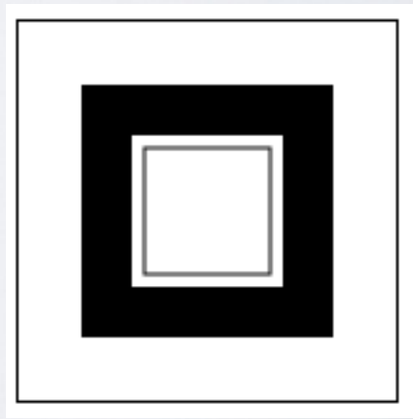
- Normally 1 unit in the grid corresponds to 1 pixel on the canvas.
- The origin of this grid is positioned in the top left corner at coordinate (0,0). All elements are placed relative to this origin. So the position of the top left corner of the blue square becomes x pixels from the left and y pixels from the top, at coordinate (x,y).
- You can translate the origin to a different position, rotate the grid and even scale it.



# Drawing Rectangles

- <canvas> only supports one primitive shape: rectangles. All other shapes must be created by combining one or more paths, lists of points connected by lines.
- There are three functions that draw rectangles on the canvas:
  - `fillRect(x, y, width, height)`: draws a filled rectangle.
  - `strokeRect(x, y, width, height)`: draws a rectangular outline.
  - `clearRect(x, y, width, height)`: clears the specified rectangular area, making it fully transparent.
- Each of these three functions takes the same parameters. x and y specify the position on the canvas (relative to the origin) of the top-left corner of the rectangle. **`width`** and **`height`** provide the rectangle's size.

```
ctx.fillRect(25, 25, 100, 100);
ctx.clearRect(45, 45, 60, 60);
ctx.strokeRect(50, 50, 50, 50);
```



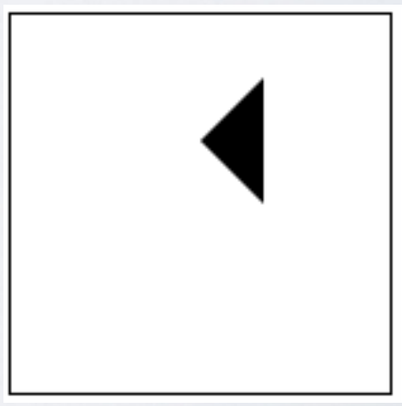
# Drawing Path

- A path is a list of points, connected by segments of lines that can be of different shapes, curved or not, of different width and of different color.
- A path, or even a subpath, can be closed.
- To make shapes using paths:
  - Create the path.
  - Draw into the path.
  - Stroke or fill the path to render it.
- The functions used to perform these steps are:
  - `beginPath()`: creates a new path. Once created, future drawing commands are directed into the path and used to build the path up.
  - `closePath()`: path method to add a straight line to the path, going to the start of the current sub-path.
  - `stroke()`: path method to draws the shape by stroking its outline.
  - `fill()`: path method to draw a solid shape by filling the path's content area.

# Moving the Pen

- The `moveTo()` function doesn't actually draw anything but it is very useful to draw paths.
- `moveTo(x, y)`: moves the pen to the coordinates specified by x and y.
- The `moveTo()` function can be used when the canvas is initialized or beginPath() is called to place the starting point or to draw unconnected paths.

```
ctx.beginPath();
ctx.moveTo(75, 50);
ctx.lineTo(100, 75);
ctx.lineTo(100, 25);
ctx.fill();
```



# Lines

- For drawing straight lines, use the `lineTo()` method.
- `lineTo(x, y)`: draws a line from the current drawing position to the position specified by x and y.
- The starting point is dependent on previously drawn paths, where the end point of the previous path is the starting point for the following.
- The starting point can be changed by using the `moveTo()` method.

# Arcs

- To draw arcs or circles you can use the `arc()` or `arcTo()` methods.
- `arc(x, y, radius, startAngle, endAngle, anticlockwise)`, draws an arc:
  - centered at (x, y) position with radius r
  - the startAngle and endAngle parameters define the start and end points of the arc in radians. These are measured from the x axis.
  - the anticlockwise parameter is a Boolean value which, when true, draws the arc anticlockwise; otherwise, the arc is drawn clockwise (defaulting to clockwise).
- `arcTo(x1, y1, x2, y2, radius)`: draws an arc with the given control points and radius, connected to the previous point by a straight line.

# Arcs Example

```
ctx.beginPath();
ctx.arc(75, 75, 50, 0, Math.PI * 2, true);
ctx.moveTo(110, 75);
ctx.arc(75, 75, 35, 0, Math.PI, false);
ctx.moveTo(65, 65);
ctx.arc(60, 65, 5, 0, Math.PI * 2, true);
ctx.moveTo(95, 65);
ctx.arc(90, 65, 5, 0, Math.PI * 2, true);
ctx.stroke();
```

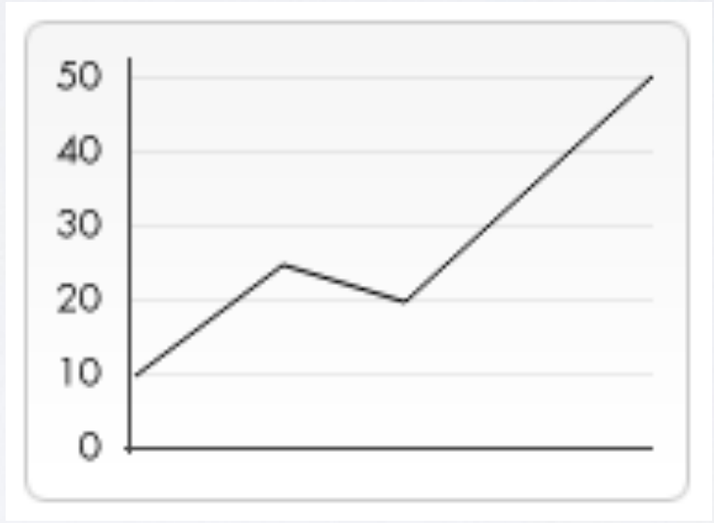
# Using Images

- The <canvas> element offers the ability to use images: these can be used to do dynamic photo compositing or as backdrops of graphs, for sprites in games, and so forth.
- External images can be used in any format supported by the browser, such as PNG, GIF, or JPEG.
- Importing images into a canvas is basically a two step process:
  - Get a reference to an HTMLImageElement object or to another canvas element as a source. It is also possible to use images by providing a URL.
  - Draw the image on the canvas using the `drawImage()` function.

# Drawing an Image

- Once you have a reference to your source image object you can use the `drawImage()` method to render it to the canvas.
- `drawImage(image, x, y)`: draws the CanvasImageSource specified by the image parameter at the coordinates (x, y).

```
var img = new Image();
  img.onload = function() {
    ctx.drawImage(img, 0, 0);
    ctx.beginPath();
    ctx.moveTo(30, 96);
    ctx.lineTo(70, 66);
    ctx.lineTo(103, 76);
    ctx.lineTo(170, 15);
    ctx.stroke();
  };
img.src = 'https://mdn.mozillademos.org/files/5395/backdrop.png';
```




# Saving and Restoring State

- `save()`: saves the entire state of the canvas.
- `restore()`: restores the most recently saved canvas state.
- Canvas states are stored on a stack. Every time the `save()` method is called, the current drawing state is pushed onto the stack.
- A drawing state consists of:
  - The transformations that have been applied (i.e. translate, rotate and scale).
  - The current values of some attributes associated to the style.
  - The current clipping path (clipping path is like a normal canvas shape but it acts as a mask to hide unwanted parts of shapes).
- You can call the `save()` method as many times as you like.
- Each time the `restore()` method is called, the last saved state is popped off the stack and all saved settings are restored.

# Translate

- `translate()` is a transformation method used to move the canvas and its origin to a different point in the grid.
- `translate(x, y)`: moves the canvas and its origin on the grid. x indicates the horizontal distance to move, and y indicates how far to move the grid vertically.
- It's a good idea to save the canvas state before doing any transformations, so you can call the restore method than having to do a reverse translation to return to the original state.

# Rotating

- `rotate()` is a transformation method that we use to rotate the canvas around the current origin.
- `rotate(angle)`: rotates the canvas clockwise around the current origin by the angle number of radians.
- The rotation center point is always the canvas origin. To change the center point, you need to move the canvas by using the `translate()` method.

# Basic Animations

These are the steps you need to take to draw a frame:

- Clear the canvas: clear any shapes that have been drawn previously. The easiest way to do this is using the `clearRect()` method.
- Save the canvas state: if you're changing any setting (such as styles, transformations, etc.) which affect the canvas state and you want to make sure the original state is used each time a frame is drawn, you need to save that original state.
- Draw animated shapes: the step where you do the actual frame rendering.
- Restore the canvas state: if you've saved the state, restore it before drawing a new frame.

# Controlling an Animation

- The `window.setInterval()`, `window.setTimeout()`, and `window.requestAnimationFrame()` functions can be used to call a specific function over a set period of time:
  - `setInterval(function, delay)`: starts repeatedly executing the function specified by function every delay milliseconds.
  - `setTimeout(function, delay)`: executes the function specified by function in delay milliseconds.
  - `requestAnimationFrame(callback)`: tells the browser that you wish to perform an animation and requests that the browser call a specified function to update an animation before the next repaint.
- If you don't want any user interaction you can use the setInterval() function which repeatedly executes the supplied code.
- You can use keyboard or mouse events to control the animation and use setTimeout().

# Animation Example

https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API/Tutorial/Basic_animations

https://developer.mozilla.org/en-US/docs/Games/Tutorials/2D_Breakout_game_pure_JavaScript

https://www.kongregate.com/games/Infernet89/have-you-missed-the-tutorial-for-life

# Questions?